OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Ставьте “+” в чат, если да
Напишите, если есть проблемы

Асинхронное и многопоточное программирование с корутинами

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	DevOps	Архитек- тура		
Модуль 4: Разработка API	OpenAPI	CollabAPI	Модели и маперы	Testing	Monitoring	
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози- торий	Конкурен- ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль- тация	Защита			

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
группе телеграм

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Цели вебинара

1. Использовать suspend-функции по назначению
2. Создавать корутины и управлять ими
3. Использовать корутины для асинхронной разработки

Маршрут вебинара

- Основы конкуренции
- Корутины и suspend-функции
- Билдеры корутин
- Structured concurrency
- Рефлексия

Многозадачность

Виды многозадачности

многозадачность

вытесняющая
(потоки)

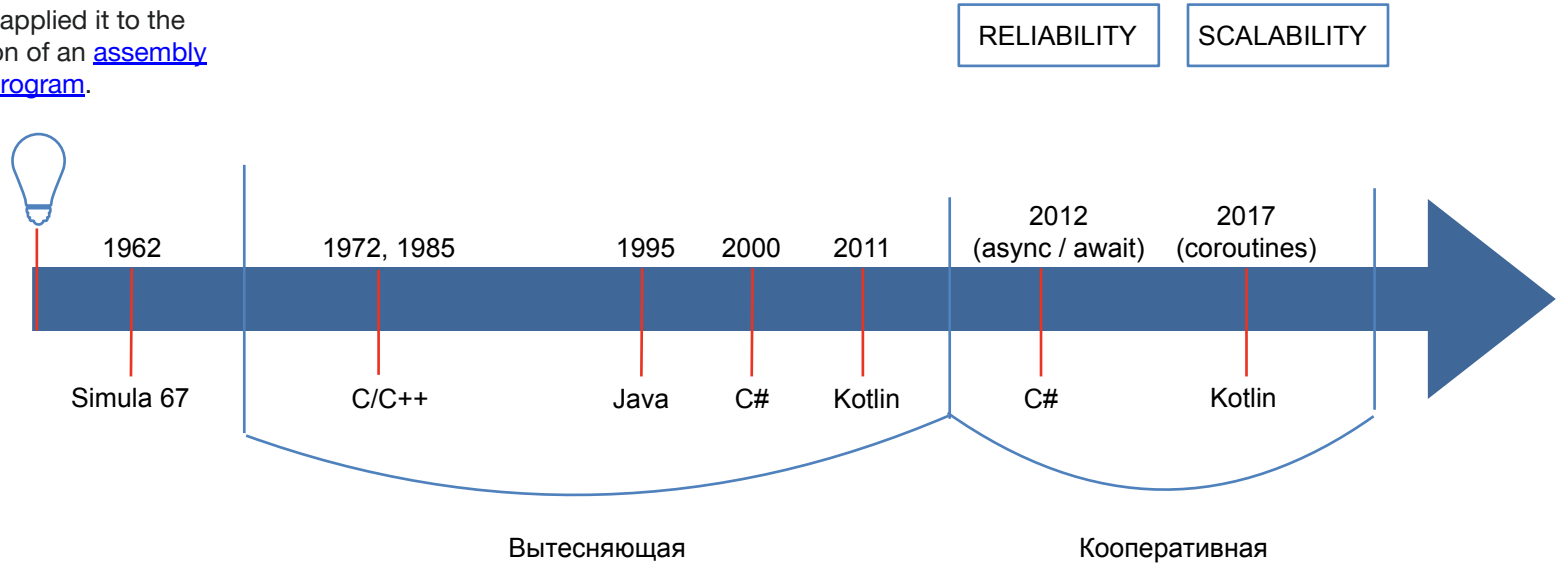
- ОС решает когда забрать управление
- квазипараллелизм на уровне ОС
- гибкое распределение ресурсов CPU
- нет риска захвата управления одной задачей

кооперативная
(асинхронность)

- задача сама отдает управление
- квазипараллелизм на уровне приложения
- отсутствует переключение контекста
- отсутсвуют системные вызовы
- более простой планировщик

История

Donald Knuth, Melvin Conway coined the term *coroutine* in 1958 when he applied it to the construction of an assembly program.



Кооперативная многозадачность

(асинхронное выполнение)

Синхронный код

```
fun getUsers(): List<User> {
    //retrieves users from client, transform to dto
    val users = httpClient.loadUsers()
    return users // returns when done
}

fun main() {
    println("Start")
    val users = getUsers()
    println("Found users $users")
}
```

Callback

```
fun getUsersAsync(callback: (List<User>) -> Unit) {
    // retrieves users from client, invoke callback when done
    async {
        val users = httpClient.loadUsers()
        callback(users)
    }
    // returns immediately
}
fun main() {
    println("Start")
    getUsersAsync { users ->
        println("Found users $users")
    }
    println("Done")
}
```

Проблемы:

- Тяжело читать
- Обработка и передача ошибок

вызывает неудобства

```
function hell(win) {
 // for listener purpose
 return function() {
   loadLink(win, REMOTE_SRC+'/assets/css/style.css', function() {
     loadLink(win, REMOTE_SRC+'/lib/async.js', function() {
       loadLink(win, REMOTE_SRC+'/lib/easyXDM.js', function() {
         loadLink(win, REMOTE_SRC+'/lib/json2.js', function() {
           loadLink(win, REMOTE_SRC+'/lib/underscode.min.js', function() {
             loadLink(win, REMOTE_SRC+'/lib/backbone.min.js', function() {
               loadLink(win, REMOTE_SRC+'/dev/base_dev.js', function() {
                 loadLink(win, REMOTE_SRC+'/assets/js/deps.js', function() {
                   loadLink(win, REMOTE_SRC+'/src/' + win.loader_path + '/loader.js', function() {
                     async.eachSeries(SCRIPTS, function(src, callback) {
                       loadScript(win, BASE_URL+src, callback);
                     });
                   });
                 });
               });
             });
           });
         });
       });
     });
   });
 };
}
```

Future/Promise/Reactive libs

```
fun getUsersAsync(): Future<List<User>> {
    // retrieves users from client, invoke callback when done
    val future = Future<List<User>>()
        async {
            val users = httpClient.loadUsers()
            future.data = users
            future.status = "SUCCESS"
        }
    // returns immediately
    return future
}


fun main() {
   getUsersAsync()
      .thenCompose { users -> println("Found users $users") }
      .onError { println("Some error occur") }

}
```

Проблемы:

- Много операторов в разных либах
- Callback все еще в наличии

Coroutines

```
suspend fun getUsers(): List<User> {
    // retrieves users from client, transform to dto
    val users = httpClient.loadUsers()
    return users // returns after suspend and received all
}

suspend fun main() {
    val users = getUsers()
    println("Found users $users")
}
```

- Асинхронный код в том же стиле, что и синхронный
- Возможно приостановить процесс вычислений
- Структурная обработка исключений
- Эффективное использование CPU и потоков за счет исключения блокировки потоков

Вытесняющая многозадачность

(Многопоточность, мультипроцессинг)

Процессы

Процесс

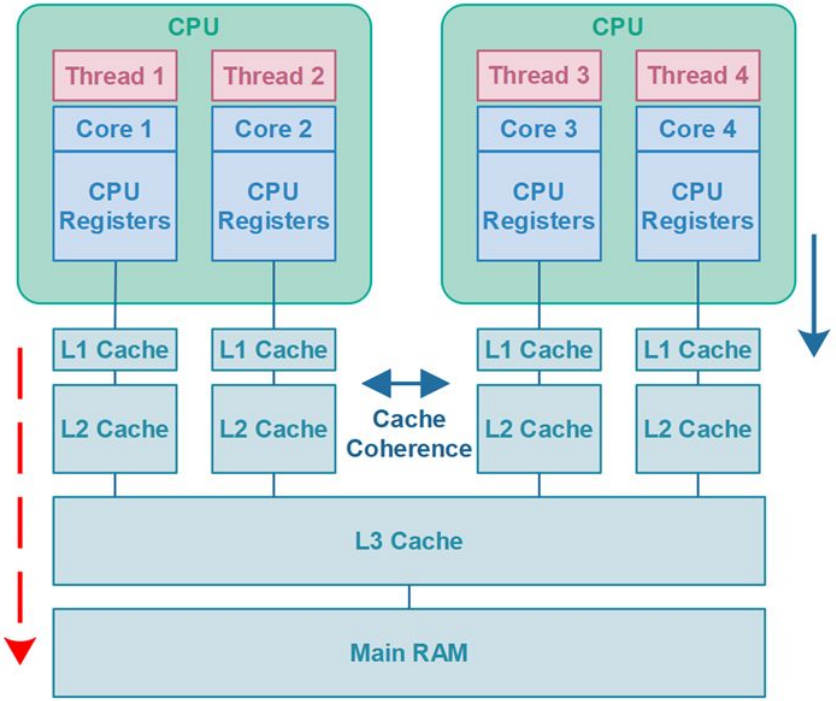
- На уровне CPU и ОС
- Разделение процессорного времени, ядер и CPU
- Полная изоляция между процессами: свои данные, стек и кэш
- Обмен только через механизмы межпроцессного взаимодействия на уровне ядра OS

Inter Process Communication:

- Unix Sockets
- Internet Sockets
- Named Pipelines (FIFO)
- Signals (SIGKILL, SIGQUIT, SIGHUP, etc)
- System V IPC
  - IPC Messages (Message Queue)
  - Semaphores
  - Shared Memory

Method	100 Byte Messages	1 Kilo Byte Messages
Unix Signals	--broken--	--broken--
ZeroMQ (TCP)	24,901 msg/s	22,679 msg/s
Internet sockets (TCP)	70,221 msg/s	67,901 msg/s
Domain sockets	130,372 msg/s	127,582 msg/s
Pipes	162,441 msg/s	155,404 msg/s
Message Queues	232,253 msg/s	213,796 msg/s
FIFOs (named pipes)	265,823 msg/s	254,880 msg/s
Shared Memory	4,702,557 msg/s	1,659,291 msg/s
Memory-Mapped Files	5,338,860 msg/s	1,701,759 msg/s

Потоки



Поток

- Особый подпроцесс в пуле системных процессов
- Свой стек и кэш, но разделяемые данные
- Обмен через общую память
- Особые проблемы из-за разделения памяти

Thread Synchronizers

– Lock
– Monitor

– Semaphore
– CountDownLatch
– CyclicBarrier
– Phaser

Thread.sleep(5) - блокирование <u>всего</u> потока

Потоки в Kotlin

```
public class Thread implements Runnable
```

```
public interface Runnable {
    public abstract void run();
}
```

```
val thread = object: Thread() {
    public override fun run() {
        println("Hello, otus!")
    }
}
```

```
class Echo: Runnable {
    override fun run() {
        print("Hello, otus")
    }
}
```

```
val thread = Thread (
        Echo()
    )
```

```
val thread = Thread(Runnable {
    println("Hello, otus!")
})
```

OR

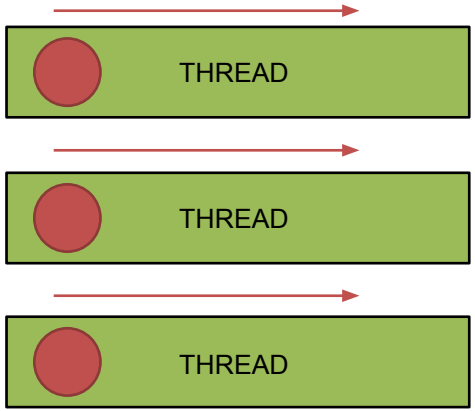
```
Echo().run() // INCORRECT
```

```
val thread = Thread {
    println("Hello, otus!")
}
```

```
thread.start()
thread.join()
```

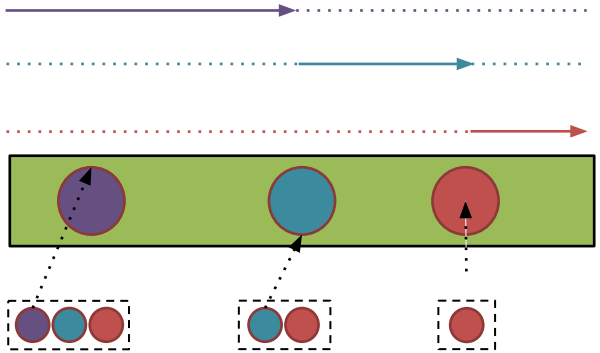
Разница между алгоритмами многозадачности

Threads



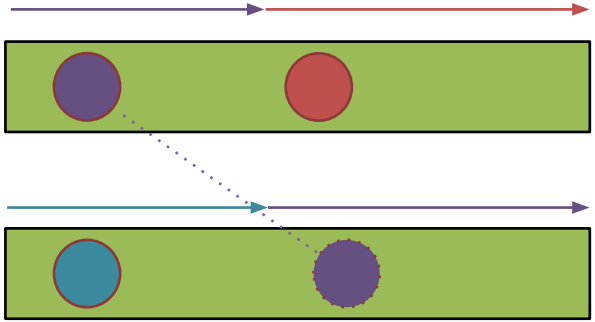
3 задачи на
3 потоках

Future



3 задачи делят ресурсы
на 1 потоке

Coroutines



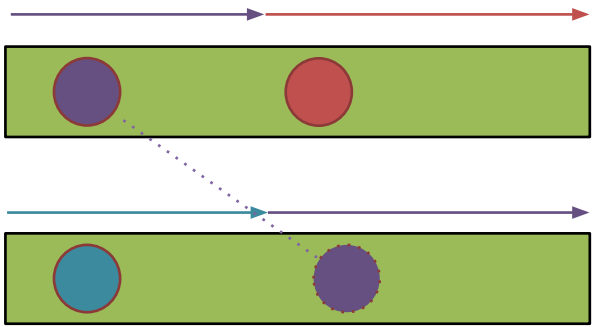
3 задачи делят 2 потока

Особенность Корутин

При штатной блокировке потока средствми JVM, останавливаются все задачи корутин, которые выполнялись на этом потоке.

Основная проблема - выявить блокировки потоков и их корректно обработать

Coroutines



3 задачи делят 2 потока

Корутины

Корутина (coroutine, сопрограмма) — компонент программы, обобщающий понятие подпрограммы, который поддерживает множество входных/выходных точек, остановку и продолжение выполнения с сохранением определённого состояния

Синтаксис. suspend-функция

```
// определение suspend-функции
suspend fun foo() {
   delay(1000)
}
```

```
// определение suspend-лямбды
val lambda = suspend {
   ...
}
```

```
// suspend-функция как аргумент в сигнатуре
fun bar(block: suspend () -> Int) {
   ...
}
```

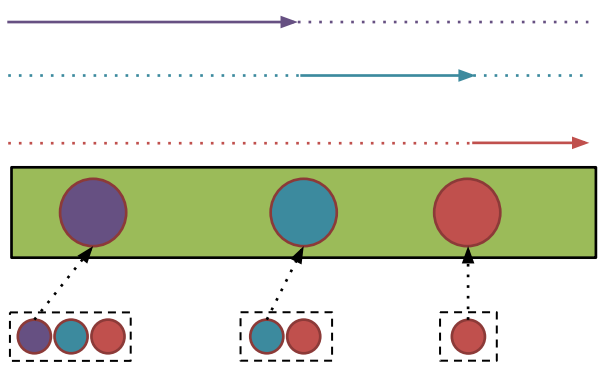
```
// использование функции
// с аргументом suspend-функцией
bar {
   ...
}
```

Continuation

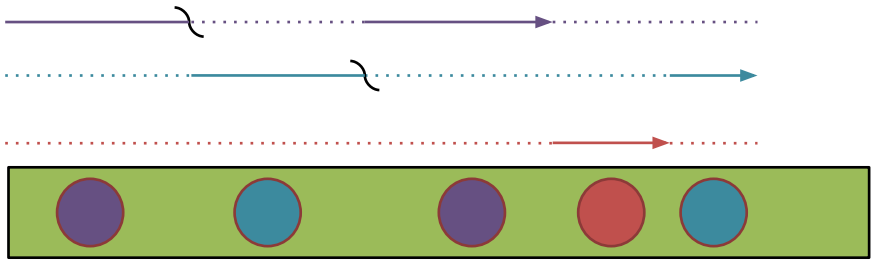
```
interface Continuation<in T> {
    // The context of the coroutine that
    // corresponds to this continuation.
    val context: CoroutineContext

    // Resumes the execution of the corresponding coroutine
    // passing a successful or failed [result]
    // as the return value of the last suspension point
    fun resumeWith(result: Result<T>)
}
```

Async



Corutines



Создание и запуск корутин

CoroutineContext

Состоит из:

Job — управляет жизненным циклом корутины (рассмотрим позже)

CoroutineDispatcher — управляет распределением работ по потокам

CoroutineName — имя корутины, полезно для отладки

CoroutineExceptionHandler — обрабатывает неотловленные исключения

Запуск из блокирующего кода

runBlocking – coroutine-билдер, используемый в качестве моста между блокирующим кодом и корутинами

Возвращает результат выполнения заданного блока

Захватывает поток

Можно использовать для main-функции, тест-функций и т.п.

Доступен на платформах JVM и Native

Недоступен в JS

```
public fun <T> runBlocking(
    context: CoroutineContext = EmptyCoroutineContext,
    block: suspend CoroutineScope.() -> T
): T
```

```
fun main(): Unit = runBlocking {
   println("Hello")
   delay(1000)
   println("World")
}
```

context: CoroutineContext

Запуск “параллельной” задачи

launch – неблокирующий coroutine-билдер, для создания и запуска корутин

Не захватывает поток

Не прерывает корутину

Может быть вызван только из области корутины, например из runBlocking или GlobalScope

```
public fun CoroutineScope.launch(
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> Unit
): Job
```

```
val job = GlobalScope.launch {
    ...
}
```

context: CoroutineContext

Запуск “параллельной” задачи

<u>async</u> – неблокирующий coroutine-билдер, для создания и запуска корутин, возвращающих значение

Не захватывает поток

Не прерывает корутину

```
public fun <T> CoroutineScope.async(
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> T
): Deferred<T>
```

```
val deferred = GlobalScope.async {
    ...
}
```

context: CoroutineContext

Смена контекста выполнения

withContext – coroutine-билдер для создания и запуска корутин, контекст которых получен из слияния текущего и заданного аргументом контекстов

Возвращает результат выполнения заданного блока

Прерывает корутину

Ожидает завершения созданной корутины

```
public suspend fun <T> withContext(
    context: CoroutineContext,
    block: suspend CoroutineScope.() -> T
): T
```

```
withContext(CoroutineName("My awesome coroutine")) {
    ...
}
```

Ограничение области выполнения

coroutineScope – coroutine-билдер для создания и запуска корутин в ограниченной области выполнения

```
suspend fun <R> coroutineScope(
    block: suspend CoroutineScope.() -> R
): R
```

Возвращает результат выполнения заданного блока

```
coroutineScope {
    ...
}
```

Прерывает корутину

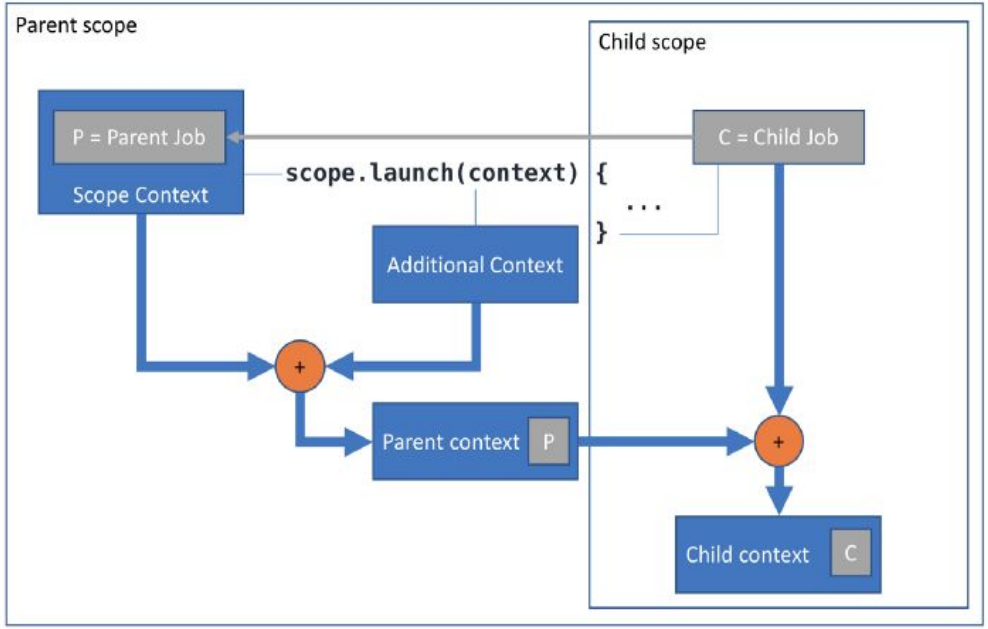
Ожидает завершения созданной корутины

Scope vs Context

Structured concurrency
каждая корутина стартуют в определенном скоупе.
Например, GlobalScope

Scope
- корутины стартуют в скоупе;
- скоуп контролирует запущенные в нем корутины, контролирует жизненный цикл;
- скоупы получают необработанные исключения;
- привязывают корутины к жизненному циклу приложения (как в Андройд);
- обеспечивает контроль за утечкой корутин.

Context
- хранит данные корутин;
- обеспечивает создание дочерних контекстов;
- формирует иерархию корутин;
- привязывает диспетчеры.



Coroutine-билдеры

- **runBlocking** - из блокирующего в корутины
- **launch** - запускает не блокируя и не прерывая
- **async** - запускает не блокируя и не прерывая, можно получить результат
- **withContext** - позволяет сменить контекст
- **coroutineScope** - создает новую ограниченную область выполнения

Coroutine-билдеры

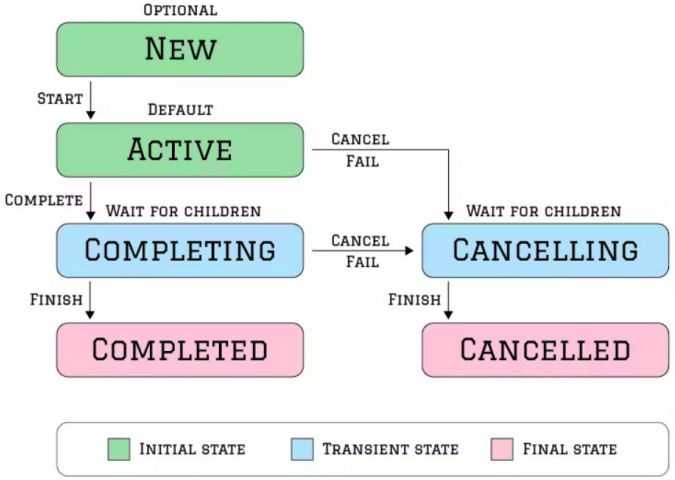
Функция	Описание	Особенности
runBlocking	Блокирует текущий поток (например, основной поток) до завершения всех корутин внутри блока	- Используется для запуска корутин в синхронном коде - Обычно используется в main-функции и тестах - Может быть использован только в блокирующем коде
launch	Запускает новую корутину, которая выполняется параллельно с основной программой	- Неблокирующий - Возвращает объект Job, который можно использовать для отслеживания состояния корутины. - Не возвращает значение
async	Запускает асинхронную корутину, которая выполняет задачу и возвращает результат через Deferred<T>	- Неблокирующий - Возвращает объект Deferred<T>, который можно использовать для получения результата с помощью await() - Возвращает результат
withContext	Переключает выполнение на новый контекст (например, на другой поток) для текущей корутины	- Меняет контекст выполнения текущей корутины (например, переключает поток) - Блокирует выполнение текущей корутины до завершения блока - Возвращает результат блока
coroutineScope	Создаёт область, в которой все корутины должны завершиться до выхода из этой области	- Ожидает завершения всех корутин в своей области - Используется для обеспечения структурированной конкуренции (all coroutines must complete) - Прерывает все корутины в случае исключения

Practice coroutines, structured concurrency

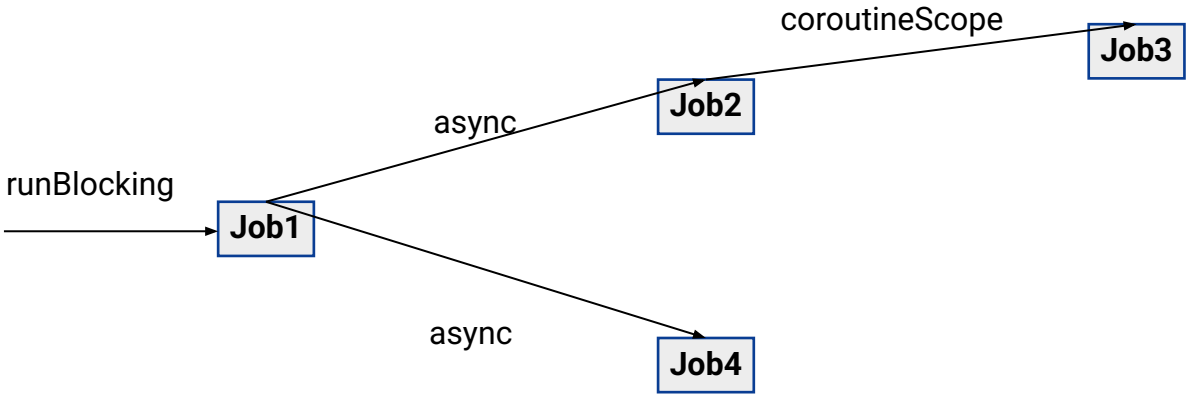
Structured concurrency

Жизненный цикл корутины: состояния Job/Deferred

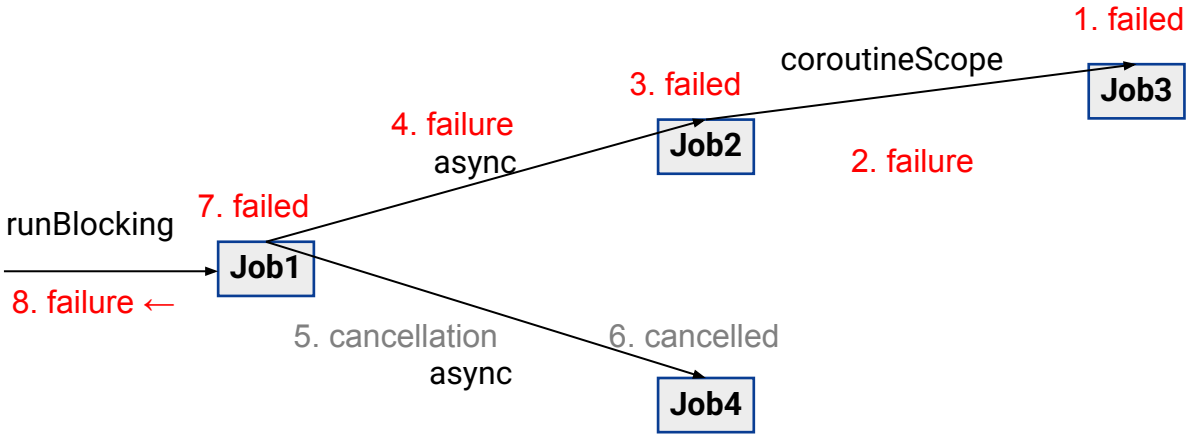
State	isActive	isCompleted	isCancelled
New	*false*	*false*	*false*
Active	*true*	*false*	*false*
Completing	*true*	*false*	*false*
Cancelling	*false*	*false*	*true*
Cancelled	*false*	*true*	*true*
Completed	*false*	*true*	*false*



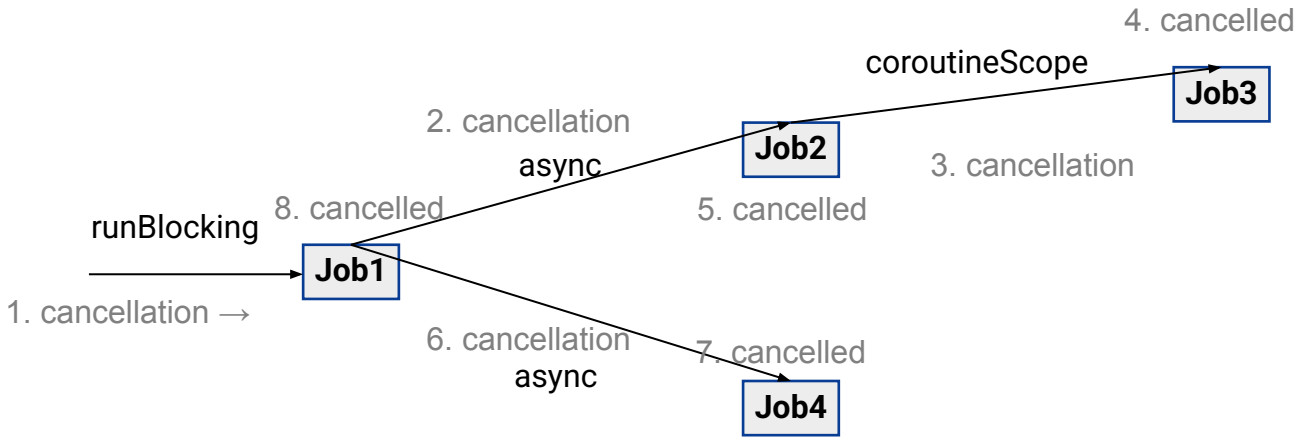
Пример иерархии Job’ов



Пример распространения ошибки

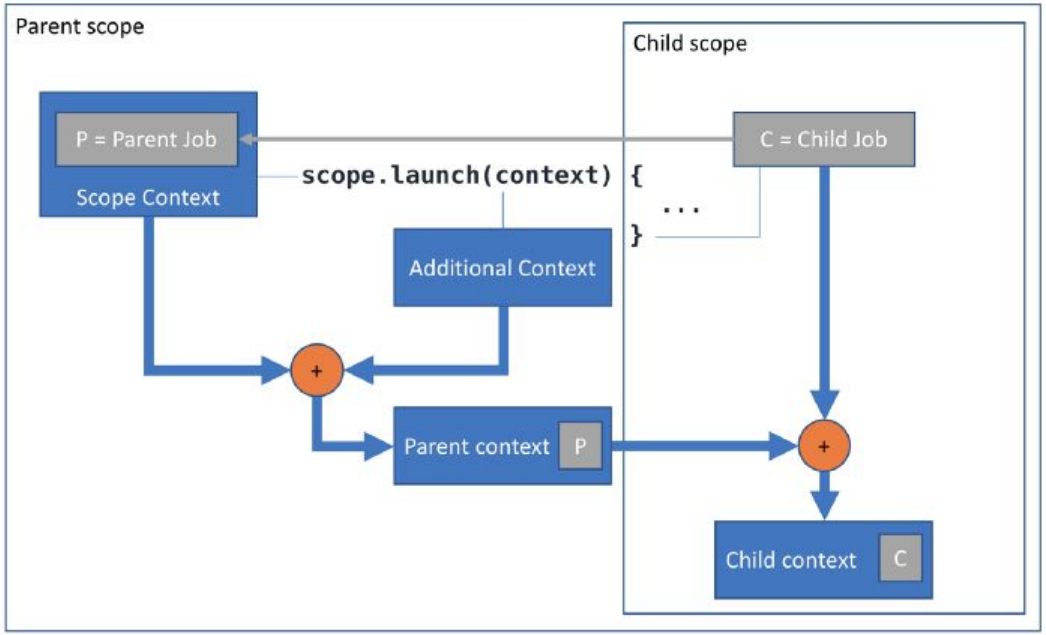


Пример распространения отмены



Формирование дочернего контекста

- Родительский контекст = значения по-умолчанию + текущий (унаследованный CoroutineContext) + аргументы
- Создается дочерняя задача Job
- Контекст новой корутины = родительский контекст + дочерняя Job



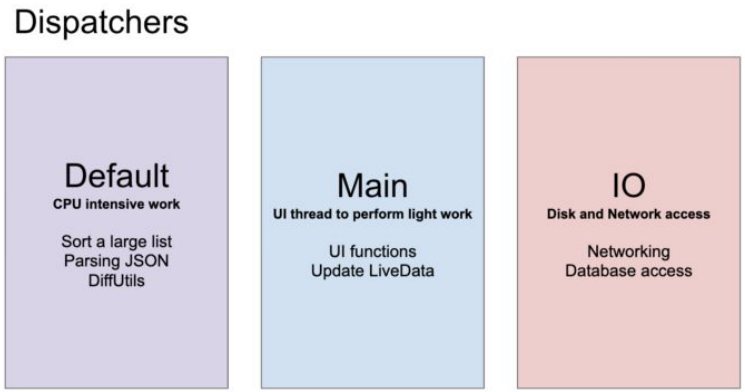
Параллелизм

Диспетчеры

Диспетчер корутин:

- распределяет корутины по потокам;

- **Dispatchers.Default** - для выполнения по умолчанию
- **Dispatchers.IO** - для выполнения блокирующего кода
- **Dispatchers.Main** - для безопасной работы с GUI
- **Dispatchers.Unconfined -** без ограничения потоком
- **asCoroutineDispatcher** - если нужен свой пул потоков
- **newSingleThreadContext, newFixedThreadPoolContext -** Создает потоки для корутин. ОПАСНО!! Требуется use



Practice Dispatchers

Рефлексия

Цели вебинара

1. Использовать suspend-функции по назначению
2. Создавать корутины и управлять ими
3. Использовать корутины для асинхронной работы

Список материалов для изучения

1. https://kotlinlang.org
2. https://gradle.org
3. https://github.com/otuskotlin/ГГГГММ-ok-marketplace
4. Гайд: https://kotlinlang.org/docs/coroutines-guide.html
5. https://github.com/Kotlin/KEEP/blob/master/proposals/coroutines.md
6. Context and Scope: https://elizarov.medium.com/coroutine-context-and-scope-c8b255d59055
7. Паттерны и антипаттерны: https://habr.com/ru/post/432942/
8. Structured Concurrency: https://www.youtube.com/watch?v=w0kfnydnFWI
9. https://kt.academy/article/cc-why
10. Exception handlilng: https://kt.academy/article/cc-exception-handling

Следующий вебинар

Модуль 2: Расширенные возможности Kotlin

Асинхронное и многопоточное программирование с каналами

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание! Приходите на следующие вебинары